

Auditchain Smart Contract Documentation - No Cohort

Overview

Auditchain protocol consist of following smart contracts

- **AuditChain**
 - Members
 - MemberHelpers
 - CohortFactory
 - DepositModifiers
 - NodeOperations
 - NodeOperationsHelpers
 - Validations
 - ValidationHelpers
 - ValidationCohort
 - ValidationNoCohort
- **Governance**
 - GovernerAlpha
 - Timelock
- **NFT**
 - RulesERC721Token
- **Token**
 - AuditToken
 - Locked

Concepts

Auditchain is a **decentralized protocol** that automates trustless financial data reporting and analysis processes that result in immutable provenance. The Auditchain protocol can be extended to automate all current and future accounting processes which have well defined repeatable steps and structured nature.

The Auditchain protocol uses decentralized storage to store the submitted financial data and the resulting analysis. To achieve the analysis, the Auditchain protocol uses financial rules, called *controls*, stored in the same immutable storage. CPAs can participate in the protocol by approving or rejecting the controls, providing peer review prior to their use. The immutable storage provides transparency and accountability for the sufficiency of the approved controls.

The Auditchain Protocol rewards rules creators with NFTs and node operators that analyze controls and financial reports with AUDT Tokens. The NFT holders are rewarded with AUDT Tokens each time their controls are used to analyze a financial data submission.

The **AUDT Token** is the underlying token of the AUDT protocol and is used for settlement and as rewards for participating in the network of financial data and control validators and staking on the protocol.

AUDT also enables its holders to govern the protocol by voting for or against proposals which programmatically affect variables within the protocol, e.g. how much a validation costs, how much is paid out to validators, etc. The weight of each vote is determined by the voter's AUDT balance. Only participants who have substantial AUDT holdings can create proposals. This ensures that the protocol is governed by those with the greatest invested interest in it.

Accountants can participate in the protocol by using its native tools and extensibility to perform their duties and receive rewards by analyzing and validating financial reports.

Enterprises can participate in the protocol by submitting reports for analysis to a group of certified public accountants and chartered accountants and receive near-instant validation of their reports. Enterprises can publish those reports to provide near-real time reporting of their financial status, bolstering trust by increasing access to their financial data.

Anyone can participate in the protocol by submitting published financial reports to the network of node operators that perform report analysis with the protocol's report analysis engine, *Pacioli*, or similar software. Submitters spend AUDT to have the financial reports analyzed, which is distributed to the node operators by the protocol. Highest performing validators receive substantially higher rewards, incentivizing node operators to build high-performance nodes so the best user experience is provided to submitters.

The Auditchain protocol uses decentralized storage to store the submitted financial data and the resulting analysis. To achieve the analysis, the Auditchain protocol uses financial rules, called controls, stored in the same immutable storage. CPAs can participate in the protocol by approving or rejecting the controls, providing peer review prior to their use. The immutable storage provides transparency and accountability for the sufficiency of the approved controls.

The Auditchain Protocol rewards rules creators with NFTs and node operators that analyze controls and financial reports with AUDT Tokens. The NFT holders are rewarded with AUDT Tokens each time their controls are used to analyze a financial data submission.

The AUDT Token is the underlying token of the AUDT protocol and is used for settlement and as rewards for participating in the network of financial data and control validators and staking on the protocol.

AUDT also enables its holders to govern the protocol by voting for or against proposals which programmatically affect variables within the protocol, e.g. how much a validation costs, how much is paid out to validators, etc. The weight of each vote is determined by the voter's AUDT balance. Only participants who have substantial AUDT holdings can create proposals. This ensures that the protocol is governed by those with the greatest invested interest in it.

Accountants can participate in the protocol by using its native tools and extensibility to perform their duties and receive rewards by analyzing and validating financial reports.

Enterprises can participate in the protocol by submitting reports for analysis to a group of certified public accountants and chartered accountants and receive near-instant validation of their reports. Enterprises can publish those reports to provide near-real time reporting of their financial status, bolstering trust by increasing access to their financial data.

Anyone can participate in the protocol by submitting published financial reports to the network of node operators that perform report analysis with the protocol's report analysis engine, Pacioli, or similar software. Submitters spend AUDT to have the financial reports analyzed, which is distributed to the node operators by the protocol. Highest performing validators receive substantially higher rewards, incentivizing node operators to build high-performance nodes so the best user experience is provided to submitters.

In conclusion, the Auditchain Protocol provides an infrastructure and native tools that allow participants to submit financial reports, analyze financial reports using immutable rules that produce immutable analyses, and provides a weighted proposal and voting system. The protocol also provides the AUDT Token that offers a means to extend the protocol, ensure its continued security and improvement and rewards participating in the protocol. Finally, the protocol provides a bridge to migrate AUDT Tokens across blockchains. Collectively, these features and their extensibility provide users with the benefits of decentralization and automation now and in the future.

Members

Overview

Introduction

Member contract role is to allow the creation of new member accounts according to 3 types.

Enterprise, Validator and Subscriber.

In addition, it maintains various parameters which are used to control internal workings of protocol like e.g. access fee, share of profits between enterprise and validators and quorum required to make validation valid. All those parameters can be updated by governance contracts through community voting. There is a special role, called "SETTER_ROLE" which is assigned to governance contract so the results of voting can be applied automatically once voting has been completed.

Global variables and types

- `uint256 public amountTokensPerValidation`

Amount of tokens per each validation which will be minted to reward validators. This only applies to cohort validations.

- `uint256 public accessFee`

This is the fee which a data subscriber will have to pay for access to existing validation of Enterprises.

- `uint256 public enterpriseShareSubscriber`

This is the enterprise percentage of share from subscriber payment to access the validation reports. In this case it is 40%. If the access fee from above is 100, then the enterprise would receive 40.

- `uint256 public validatorShareSubscriber`

This is the validator percentage of share from subscriber payment to access the validation reports. In this case it is 40%. If the access fee from above is 100, then the enterprise would receive 40.

- `address public platformAddress;`

This is the address where earnings from processing of each of the validations will be set to the platform. This is currently only implemented for the cohort model.

- `uint256 public platformShareValidation`

This is how much the platform will earn from each validation. In this case this is 15%.

- `uint256 public enterpriseMatch`

This is how much the enterprise will have to pay for each validation. E.g. if **amountTokensPerValidation** is equal to 100, then Enterprise would have to pay 200.

- `uint256 public minDepositDays`

Number of days to consider for calculation of average daily spendings. To provide notice to enterprise or data subscribers that their deposits are reaching bottom and they need to deposit more funds so their validations can continue.

- - `uint256 public requiredQuorum`

Quorum required to consider validation valid. If there are e.g. 100 validators, the system would need 80% for the transaction to be completed. When 80% of validators have validated the request, validation is considered to be completed. The remaining validator can still validate but they will not be considered for the POW earning.

- `mapping(address => mapping(UserType => string)) public user`

Mapping to store user name.

- `mapping(address => mapping(UserType => bool)) public userMap`

Mapping to determine if user and user type already exist.

- `address[] public enterprises`

List of addresses for enterprises

- `address[] public validators`
- `address[] public validators`

List of addresses for Validators

- `address[] public dataSubscribers`

List of addresses for Data Subscribers

- `enum UserType {Enterprise, Validator, DataSubscriber}`

User types

-

Events

- `event UserAdded(address indexed user, string name, UserType indexed userType) ;`

Fired when new user has been added

- `event LogGovernanceUpdate(uint256 params, string indexed action) ;`

Fired when governance contract updates one of the parameters

Modifiers

- `modifier isSetter`

Will ensure that function is called by a caller who has credentials of **SETTER_ROLE**

- `modifier isController`

Will ensure that function can be only executed if caller is a controller which has credentials of **CONTROLLER_ROLE**

○

Functions

- `function initialize(address _auditToken, address _platformAddress)
initializer external`

○ `auditToken = IAuditToken(_auditToken) ;`

○ `platformAddress = _platformAddress ;`

- `amountTokensPerValidation = 1e18;`
 - `accessFee = 1000e18;`
 - `enterpriseShareSubscriber = 40;`
 - `validatorShareSubscriber = 40;`
 - `platformShareValidation = 15;`
 - `enterpriseMatch = 200;`
 - `minDepositDays = 30;`
 - `requiredQuorum = 80;`
 - **Creator of the contract receives the admin role.**
- `function addUser(address newUser, string memory name, UserType userType) external isController()`

Add a new user. To create a new user we need to pass user type, which is **enterprise**, **validator** or **data subscriber**, user address and the name.

Governance update functions

- `function updateQuorum(uint256 _requiredQuorum) external isSetter()`
- `function updatePlatformShareValidation(uint256 _newFee) external isSetter()`
- `function updateAccessFee(uint256 _accessFee) external isSetter()`
- `function updateMinDepositDays(uint256 _minDepositDays) external isSetter()`
- `function updateTokensPerValidation(uint256 _amountTokensPerValidation) external isSetter()`
- `function updateEnterpriseMatch(uint256 _enterpriseMatch) external isSetter()`
- `function updateDataSubscriberShares(uint256 _enterpriseShareSubscriber, uint256 _validatorShareSubscriber) external isSetter()`

Member Helpers

Overview

Introduction

It contains additional functions for Members pertaining to the financial side of their membership. Those are functions to stake and redeem their amounts, also functions which are used by other contracts which modify/increase users holdings due to participation in validations as validators or decrease users holding if they are acting as enterprises or data subscribers and are requesting validations and payments to validators are issued.

Global variables and types

- `uint256 public totalStaked`

Total amount of staked funds

- `uint256 public minContribution`

Minimum stake amount for Validators

- `uint256 public maxContribution`

Maximum contribution for Validators

- `mapping(address => uint256) public deposits`

Values of deposits for each users

- `IValidations public validations`

Validation contract

- `Members public members`

Member contract

- `address public auditToken`

Token contract

Events

- `event LogDepositReceived(address indexed from, uint256 amount)`

Fired when deposit has been received

- `event LogDepositRedeemed(address indexed from, uint256 amount)`

Fired when deposit has been redeemed

- `event LogIncreaseDeposit(address user, uint256 amount)`

Fired when deposit has been increased

- `event LogDecreaseDeposit(address user, uint256 amount)`

Fired when deposit has been decreased

Modifiers

- `modifier isValidator(string memory source)`

Will ensure that function is called by valid user who is Validator

- `modifier isController(string memory source)`

Will ensure that function can be only executed if caller is a controller which has credentials of `CONTROLLER_ROLE`

Functions

- `function initialize(address _members, address _auditToken) external`
 - `minContribution = 5e21;`
 - `maxContribution = 25e21;`
 - `members = Members(_members);`
 - `auditToken = _auditToken;`
 - `_setupRole(DEFAULT_ADMIN_ROLE, msg.sender)`

- `function stake(uint256 amount) external nonReentrant`

Validators, Enterprises and Data Subscribers need to deposit funds in order to operate the system. Stake function is responsible for transferring funds from the user account into the contract.

- `function redeem(uint256 amount) external nonReentrant`

This function is mostly dedicated to Validators who will be able to redeem their funds from validations. But any user can remove their remaining funds.

- `function setValidation(address _validations) external
isController("setValidation")`

This is a setter function which is only responsible for updating a function `setValidation(address _validations)` contract with an address of the Validation contract, which is responsible for performing validations.

- `function increaseDeposit(address user, uint256 amount) external
isController("increaseDeposit") returns(bool)`

This function is mostly used by other contracts to increase deposit amounts for validators when they earn funds for validation.

- `function decreaseDeposit(address user, uint256 amount) external isController("decreaseDeposit") returns (bool)`

This function is mostly used by other contracts to decrease deposit amounts for Enterprises and Data Subscribers when they request validation and payment is issued to Validators.

- `function returnDepositAmount(address user) external view returns (uint256)`

This function will return the deposit amount for a given user.

This function is mostly used by other contracts to decrease deposit amounts for Enterprise and Data Subscribers when they have to pay for validations

.

Cohort Factory

Overview

Introduction

Allows the creation of invitations by Enterprise to Validators and acceptance of those invitations by Validators. Then Enterprise can create a cohort consisting of invited Validators if their number is larger or equal to value set in the contract.

Global variables and types

- `enum AuditTypes {Unknown, Financial, System, NFT, Type4, Type5}`

Types of audits and their names.

- `bytes32 public constant SETTER_ROLE = keccak256("SETTER_ROLE")`

A role needed to execute change of parameters in a smart contract resulting from governance calls.

- `uint256[] public minValidatorPerCohort`

An array to hold the initial minimum of validators per cohort.

- ```
struct Invitation {
 // address enterprise;
 address validator;
 uint256 invitationDate;
 uint256 acceptanceDate;
 AuditTypes audits;
 // address cohort;
 bool deleted;
}
```

Invitation structure to hold info about its status

- `mapping(address => mapping(uint256=>bool)) public cohortMap`

Mapping to store cohort list for each Enterprise

- `mapping (address => mapping(address=> AuditTypes[])) public validatorCohortList`

Map of validators and cohorts.

## Events

- `event ValidatorInvited(address inviting, address indexed invitee, AuditTypes indexed audits, uint256 invitationNumber);`

Fired when new validator has been invited

- `event InvitationAccepted(address indexed validator, uint256 invitationNumber);`

Fired when invitation has been accepted

- `event CohortCreated(address indexed enterprise, uint256 audits);`

Fired when cohort has been created

- `event UpdateMinValidatorsPerCohort(uint256 minValidatorPerCohort, AuditTypes audits)`

Fired when min validators per cohort has been updated

- `event ValidatorCleared(address validator, AuditTypes audit, address enterprise)`

Fired when the validator has been removed or invitation canceled from the cohort.

## Functions

- `function initialize(address _members, address _memberHelpers) external`

- **Members** - address of the members contract where the information about the user types is stored.
- **Minimum validators per cohort** - initial number set to [3,3,3,3,3] for 6 types of cohorts.
- **Creator of the contract receives the admin role.**

- `function inviteValidator(address validator, uint256 audit) external`

Enterprise (in other words party requesting validations) can create cohorts currently in 3 groups. **Financial, System, NFT**. In order to create Cohort of validators, Enterprise has to invite set of vlidators who already have pre registered with the platform and have undergone KYC and have valid CPA license.

- **function** `acceptInvitation(address enterprise, uint256 invitationNumber)` **public**

Once an invitation has been created, Validators have the option to accept the invitation.

- **function** `createCohort(uint8 audit)` **external**

When the invitation has been accepted, Enterprise can create a cohort. Only one cohort can be created for each cohort type.

- **function** `clearInvitationRemoveValidator(address validator, AuditTypes audit)` **external returns (bool)**

Enterprise can remove the validator from the cohort at any time during the operation.  
Enterprise can clear invitations which have not been accepted by Validators.

- **function** `returnValidatorCohortsCount(address validator, address enterprise)` **external view returns (uint256)**

Return cohorts count

- **function** `returnValidatorList(address enterprise, uint8 audit)`**public view returns(address[] memory)**

Return list of all validators

- **function** `returnCohorts(address enterprise)` **external view returns (bool[] memory)**

Return array of bool true or false for each cohort created

- `function isValidatorInvitedNumber(address enterprise, address validator, uint256 audits, uint256 invitNumber) external view returns (bool, bool)`

Determine if the validator has been invited and if the invitation has been accepted based on an invitation number.

- `function isValidatorInvited(address enterprise, address validator, uint256 audits) public view returns (bool, bool)`

Determine if the validator has been invited and if the invitation has been accepted based on enterprise address.

## Node Operations

### Overview

### Introduction

Deals with all activity relating to node operators. It allows switchin operator roles, defining pools of delegating node operators, specifying delegations, increasing rewards and claiming of those rewards.

### Global variables and types

- `MemberHelpers public memberHelpers;`

Contract with helper functions

- `address public auditToken;`

Contract with Audit token

- `Members public members;`

Contract with user account

- `address[] public nodeOperators;`

Iterable list with node operators addresses;

- `address[] public CPAs;`

List with accounts who registered as CPAs;

- `uint256 public stakeRatio;`

Number used as divider against user deposits when calculating rewards for each staking operation.

E.g. the user has 20,000 AUDT tokens and the Stake ratio is 1000.

User earnings staking earnings per validation would be

$20,000 / 1000 = 20$  AUDT tokens.

- `uint256 public stakeRatioDelegating;`

Similar to the above this is a divider for those validators who don't run their own node, but are delegating their stake to another user who is running node.

- `uint256 public stakingRatioReferral;`

Similar to the above this is the divider for those validators who run a node and have stake delegated to them. Each user who delegates stake to them will increase their earnings.

- `struct nodeOperator {`

```
 bool isNodeOperator; // true if user is node operator
```

```
 address[] delegations; // list of user who are delegating their
 // to user in question
```

```
 mapping(address => bool) isDelegating; // true if user is
 // delegating to others
```

```
 uint256 stakeAmount; // total earnings due to validations
```

```
 uint256 delegateAmount; // total earnings if delegating to other
```

```

uint256 referralAmount; // total earnings if others delegate
uint256 POWAmount; // total earnings from POW race
address delegatorLink; // if delegating than to whom
bool noDelegations; // how many members are delegating
bool isCPA; // true if user is CPA
}

```

Node operator structure holding all parameters defining state of the node operator.

- `mapping(address => nodeOperator) public nodeOpStruct;`

Mapping of node operators

- `bytes32 public constant CONTROLLER_ROLE = keccak256("CONTROLLER_ROLE");`

Controller role needed for manipulating earnings resulting from actions specified above. This role is assigned to other smart contracts which track validations. E.g. ValidationsNoCohort.sol.

- E.g. if validation has occurred, ValidationHoChort.sol will call NodeOpeations to increase e.g. POW earnings.

- `bytes32 public constant SETTER_ROLE = keccak256("SETTER_ROLE");`

Setter role is needed to assign to a governance contract, when some parameters need to be changed in the contract. E.g. Someone in the community made a proposal to increase earnings from staking for node operators. If a proposal goes through, governance contracts which has this role will automatically execute updates to smart contract.

## Events

- `event LogNodeOperatorToggled(address indexed user, string action);`

Emitted when node operator roles has been changed.

- `event LogCPAToggled(address indexed user, string action);`

Emitted when CPA role has been changed.

- `event LogNoDelegateToggled(address indexed user, string action);`

Emitted when user delegating cancels delegations.

- `event LogRemoveDelegation(address indexed delegating, address indexed delegatee);`

Emitted when delegation has been canceled by any of the parties.

- `event LogDelegation(address indexed delegating, address indexed newDelegatee);`

Emitted when delegation has been initiated.

- `event LogDelegatedStakeRewardsIncreased(address indexed delegating, uint256 amount);`

Emitted when validation occurred in the node to which user has delegated their stake.

- `event LogReferralStakeRewardsIncreased(address indexed delegating, uint256 amount);`

Emitted when user completed validation and there were other user delegating stake to his/her pool

- `event LogStakingRewardsTransferredOut(address indexed user, uint256 amount);`

Emitted when the user claims their rewards and takes them out of the platform.

- `event LogStakingRewardsClaimed(address indexed user, uint256 amount);`

Emitted when the user claims their rewards and they are increasing their stake.

- `event LogStakeRewardsIncreased(address indexed validator, uint256 amount);`

Emitted when the user has validated the report.

- `event IncreasePOW(uint256 amount, address validator);`

Emitted when the user wins the race of validations and payment is issued.

- `event LogGovernanceUpdate(uint256 params, string indexed action);`

Whenever the community updates any of the params in the contract this event is triggered.

## Modifiers

- `modifier isController(string memory source)`

Will ensure that function can be only executed if caller is a controller which has credentials of `CONTROLLER_ROLE`

- `modifier isSetter`

Will ensure that function is called by a caller who has credentials of `SETTER_ROLE`

## Functions

- `function initialize(address _memberHelpers, address _auditToken, address _members) external`

Initializes all variables for upgradable contracts. Also takes place of a typical constructor.

- `function updateStakeRatioDelegating(uint256 _newRatio) external isSetter()`

The community can change the ratio for the delegating party. This function can be only called by governance contract.

- **function** `updateStakingRatioReferral(uint256 _newRatio)` **external** **isSetter()**

The community can change the ratio for the node operator for each delegation/referral . This function can be only called by governance contract.

- **function** `updateStakeRatio(uint256 _newRatio)` **external** **isSetter()**

The community can change the stake ratio for the node operator. This function can be only called by governance contract.

- **function** `returnDelegatorLink(address operator)` **external view** **returns (address)**

This function will return the address of the pool owner to which this validator has delegated his/her tokens.

- **function** `isNodeOperator(address operator)` **external view** **returns (bool)**

It returns true if the user is a node operator, false otherwise.

- **function** `returnPoolList(address poolOperator)` **external view** **returns (address[] memory, uint256[] memory)**

This function returns users and their stake in the pool for specific pool operator/node operator.

- **function** `removeAllDelegations()` **public**

This function will remove all delegations (users in the pool) if called from the pool owner.

- **function toggleNodeOperator( ) external**  
**isValidator("toggleNodeOperator")**

This function will toggle the role of the pool operator for calling users.

- **function toggleCPA( ) external isValidator("toggleCPA")**

This function will toggle the role of the CPA for calling users.

- **function toggleNoDelegate( )**

This function will add a user to the pool or remove him/her from the pool for the calling user.

- **function removeCPA() internal**

Internal function to complete a cleanup after CPA has been removed.

- **function removeNodeOperator() internal**

Internal function to complete a cleanup after Node Operators has been removed.

- **function returnNodeOperatorsCount() external view returns(uint256)**

This function returns count of node operators

- **function returnCPAs() external view returns (address[] memory)**

This function returns the count of CPAs.

- **function removeDelegation() public isValidator("removeDelegation")**

This function will remove delegations for pool members.

- **function** `delegate(address delegatee)` **external**  
`isValidator("Delegate")`

Users can delegate their stake to another user who is a node operator.

- **function** `increasePOWRewards(address validator, uint256 amount)`  
**external** `isController("increasePOWRewards")` **returns** `(bool)`

This function will allow an increasing POW rewards for winning validators. Can only be called by contract which has the role of controller.

- **function** `increaseDelegatedStakeRewards(address validator)` **external**  
`isController("increaseDelegatedStakeRewards")` **returns** `(bool)`

This function will allow an increasing delegated stake rewards for the delegating party and for the pool operator to which tokens are delegated. Can only be called by contract which has the role of controller

- **function** `increaseStakeRewards(address validator)` **external**  
`isController("increaseStakeRewards")` **returns** `(bool)`

This function will allow an increasing stake rewards for node operator. Can only be called by contract which has the role of controller

- **function** `claimStakeRewards(bool deliver)` **external**

This function will allow users to claim their rewards. Rewards can be transferred out of contract into their wallet or increase their stake.

# Queue

## Overview

## Introduction

Deals with maintaining a pool of requests and their status during processing. When a Data Subscriber submits a request for validation, the request is placed in the queue sorted by the price offered by the submitting party. The request waits in the queue until elements in the queue preceding it are processed. The element is removed from the queue when the required quorum of validators have validated it and when the winning validator has been paid. Submitting party has the ability to increase the price of validation to move their submission to the beginning of the queue or remove it entirely from the queue before it has been processed.

## Global variables and types

- `uint256 public head;` Beginning of the queue.
- `uint256 public idCounter;` Counter of the element in the queue
- `uint256 public queueCount;` Number of the elements in the queue
- ```
struct Object{
    uint256 id;           // id of the element
    uint256 next;         // id of next element
    uint256 price;        // price for submission
    bytes32 validationHash; // hash of validation (time and doc hash)
    bytes32 documentHash; // hash of validated document
    string url;           // url of submitted document
    address user;         // requesting party
    uint256 initTime;     // time when submission was made
    bool executed;        // true if quorum has validated it
}
```
- `mapping (uint256 => Object) public objects;`

List of elements in the queue

- `bytes32 public constant CONTROLLER_ROLE = keccak256("CONTROLLER_ROLE");`

Controller role needed for setting the queue elements, and removing them. This role is assigned to other smart contracts which track validations. E.g. `ValidationsNoCohort.sol`.

E.g. If a request for validation has occurred, `ValidationHoChort.sol` will call `Queue.sol` to add a new element for all validators to see and act on it.

Events

- `event ObjectCreated(uint256 id, uint256 price, bytes32 validationHash, bytes32 documentHash, string url, address user, uint256 initTime);`

Emitted when a new queue element has been created.

- `event ObjectsLinked(uint256 prev, uint256 next);`

Emitted when two elements in the queue are linked.

- `event ObjectRemoved(uint256 id);`

Emitted when the queue element is destroyed.

- `event NewHead(uint256 id);`

Emitted when an element is added to the head.

Modifiers

- `modifier isController(string memory source)`

Will ensure that function can be only executed if caller is a controller which has credentials of `CONTROLLER_ROLE`

Functions

- **function** `get(uint256 _id)` **public virtual view**
returns `(uint256 id, uint256 next, uint256 price, bytes32 validationHash, bytes32 documentHash, string memory url, address user, uint256 initTime, bool executed)`

Will return all elements of the queue based on the queue id.

- **function** `findPrevId(uint256 _id)` **public virtual view** **returns** `(uint256)`

Will return the previous element in the queue based on the passed element id.

- **function** `findTailId()` **public virtual view** **returns** `(uint256)`

Will find the last element of the queue.

- **function** `findIdForData(uint256 _price)` **external virtual view** **returns** `(uint256)`

Will return the id of the queue element based on the price of the submission.

- **function** `findIdForValidationHash(bytes32 _validationHash)` **public view** **returns** `(uint256)`

Will return the id of the queue element based on the validation hash.

- **function** `findIdForLesserPrice(uint256 _price)` **public virtual view** **returns** `(uint256)`

Will return the id of the queue element which is less than the price passed to it.

- **function** `addHead(uint256 _price, bytes32 _validationHash, bytes32 _documentHash, string memory _url, address _user, uint256 _initTime)`

internal virtual

Will add head to the queue with passed arguments.

- **function addTail(uint256 _price, bytes32 _validationHash, bytes32 _documentHash, string memory _url, address _user, uint256 _initTime) internal virtual**

Will add a tail to the queue with passed arguments.

- **function remove(uint256 _id) internal virtual**

Will remove the first queue element.

- **function insertAfter(uint256 _prevId, uint256 _price, bytes32 _validationHash, bytes32 _documentHash, string memory _url, address _user, uint256 _initTime) internal virtual**

Will insert a new queue element after one passed to it.

- **function insertBefore(uint256 _nextId, uint256 _price, bytes32 _validationHash, bytes32 _documentHash, string memory _url, address _user, uint256 _initTime) internal virtual**

Will insert a new queue element before the element provided.

- **function _setHead(uint256 _id) internal**

It will set a new head element of the queue.

- **function _createObject(uint256 _price, bytes32 _validationHash, bytes32 _documentHash, string memory _url, address _user, uint256 _initTime) internal returns (uint256)**

Will create a new structural object which represents the basic queue element based on the provided inputs.

- **function _link(uint256 _prevId, uint256 _nextId) internal**

It will link two queue elements.

- `function addToQueue(uint256 _price, bytes32 _validationHash, bytes32 _documentHash, string memory _url, address _user, uint256 _initTime ) public isController() returns(bool)`

It will add a new element to the queue. Called by the validation contracts like **ValidationsNoCohort.sol** which has the role of the controller of this function.

- `function getValidationToVote(bytes32 _lastValidationHash) external view returns (bytes32)`

It will return validation to vote which is next after provided validation as an input.

- `function getNextValidationToVote() external view returns(bytes32 validationHash)`

It will return the next validation to vote available in the queue.

- `function getValidationToProcess(bytes32 _lastValidationHash) external view returns(bytes32 validationHash)`

It will return validation to act one which is next after provided one as an input.

- `function getNextValidation() external view returns(bytes32, bytes32, string memory, address, uint256)`

It will return the next validation to act one available in the queue.

- `function removeFromQueue(bytes32 _valHash) public isController() returns (bool)`

It will remove a given element from the queue. Called by the validation contracts like **ValidationsNoCohort.sol** which has the role

of the controller of this function.

- `function setValidatedFlag(bytes32 _valHash) external isController() returns (bool)`

It will mark the queue element as validated by the required quorum.

- `function returnQueueSize() external view returns (uint256)`

It will return the queue size at any given moment.

- `function replaceValidation(uint256 newPrice, bytes32 _valHash, bytes32 _documentHash, string memory _url, address _user, uint256 _initTime) external isController() returns (bool)`

It will replace an existing validation in the queue with the new price. If the new price is 0, the element will be removed from the queue.

ValidationHelpers

Overview

Introduction

Deals with validation process and contains helper library like functions for validation contracts like **ValildationNoCohort.sol**

Global variables and types

- `enum ValidationStatus {Undefined, Yes, No}`

Validation can be approved or disapproved. Initial status is undefined.

- `MemberHelpers public memberHelpers;`

Member helper contracts.

- `IQueue public queue;`

Queue contract.

- `mapping(address => bool) public valAddresses;`

List of contract addresses which are responsible for a validation. They have to be registered first in order to be accessed by functions in this contract.

- `bytes32 public constant CONTROLLER_ROLE = keccak256("CONTROLLER_ROLE");`

Controller role needed for setting the address of validation contracts . E.g. ValidationsNoCohort.sol.

Events

- `event ReplaceCancelValidation(address indexed user, bytes32 validationHash, uint256 price);`

Emitted when a user requests change of price of validation or removal of the validation request from the queue.

Functions

- `function initialize(address _memberHelpers, address _queue) external`

Initializes all variables for upgradable contracts. Also takes place of a typical constructor.

- `function setValAddress(address _valAddress) external`

It will allow setting the address of contracts which are responsible

for validations like `ValidationsNoCohort.sol`.

- **function** `isHashAndTimeCorrect`(`bytes32` documentHash, `uint256` _validationTime) **external view returns**

It will return true if passed inputs represent valid validation hash.

- **function** `returnWinnerStruct`(`bytes32` validationHash, `address` validationContract)**external view returns** (`string` memory valUrl, `address` winner, `uint256` validationTime)

It will return the url of the validated report, winner and validation time for a given validation hash.

- **function** `replaceCancelValidation`(`uint256` price, `bytes32` validationHash, `address` validationContract) **external**

It will call `Queue.sol` and direct replacement of an existing validation request with a new price. If the price is 0, it will direct it to remove an existing request.

- **function** `selectWinner`(`bytes32` validationHash, `address[]` memory winners) **external view returns** (`address`)

It will select the winner of the validation race, based on validation hash and array of winners. It will return the winning validator, based on the votes cast by other validators.

- **function** `determineWinners`(`bytes32` validationHash) **external view returns** (`address[]` memory, `uint256`)

It will determine who qualifies as a winner based on the consensus and time when the result of validation was saved. If more than one validator has been found for the same top validation time and they are in the consensus, the function will return an array of such validators.

- **function insertionSort(bytes32 validationHash) internal view returns (address[] memory, uint256[] memory, uint256[] memory)**

It will sort all validators who validated the report according to the time when the validation occurred.

- **function determineConsensus(uint256[] memory validation) public pure returns(uint256)**

It will analyze all votes and return consensus. It can be either valid or invalid. If there is a tie, consensus will be invalid.

- **function calculateVoteQuorum(bytes32 validationHash, address validationContract)external view returns (uint256)**

It will return the current participation level of votes taking into consideration staked tokens for each validator.

E.g. if total stacked tokens for current eligible validators is 1000 and 800 has been used to vote, the quorum would be 80. It is not important how many validators own these 1000 tokens. It could be one owing 990 and 10 others owing 10.

ValidationsNoCohort

Overview

Introduction

Deals with validations of existing reports which are public and have been already filed with regulatory bodies. It also can be used to validate process controls which are used to verify reports. Data Subscriber or rule creator (process control creator) can create submission for validation. Submission requires providing an url of the submitted content and the price submitting party is willing to pay for validation to a group of validators who are running validation nodes and are considered to be node operators. Validators run nodes which consists of two major components,

1. **Pacioli Validation Engine** - a Prologue validation tool which understands how to validate financial reports using XBRL as its underlying information exchange language.

2. **Pacioli Agent** - Node.js program utilizing Web3 libraries which invokes Pacioli Engine when a new request for validation is submitted. Once Pacioli Validation Engine returns the validation result, Pacioli Agent saves this result on IPFS and in the blockchain. To encourage quality of the validators work, protocol implements proof of work concepts and staking rewards. Validator who is determined by the network to be the winning validator receives a bounty, while all validators who participated in the process receive staking rewards.

ValidationNoCohort smart contracts implements functionality which accepts requests for submission, subsequently accepts saved results from each validator and implements a voting process where each validator verifies each other in respect to consensus and time when validation occurred. Winning validator emerges from this verification process and is paid a bounty when the process has been finished. All other validators receive staking rewards.

Global variables and types

- `IMembers public members; // members contract`
- `IQueue public queue; // queue contract`
- `IMemberHelpers public memberHelpers; // member helpers contract`
- `INodeOperations public nodeOperations; // node operations contract`
- `IValidationHelpers public validationHelpers; // validation helpers`

- `mapping(address => uint256) public outstandingValidations;`

Validations which are placed in the queue but haven't completed yet.

- `enum ValidationStatus { Undefined, Yes, No }`

Validation can be approved or disapproved. Initial status is undefined.

- `struct Validation {`
 `bool cohort; //true if this is cohort type`
 `address requestor; // address of requester`
 `uint256 validationTime; // time when request has been entered`
 `uint256 executionTime; // time when request has been completed`
 `string url; // url of the request`
 `uint256 consensus; // consensus of the validation`
 `uint256 validationsCompleted; // number of validations so far`
 `mapping(address => ValidationStatus) validatorChoice;`
 `// choice of each validator`
 `mapping(address => uint256) validatorTime;`
 `// time of each validator`
`}`

```

mapping(address => string) validationUrl;
                                // location of each validator's report
mapping(address => uint256) winnerVotesPlus;
                                // positive votes for each validator
mapping(address => uint256) winnerVotesMinus;
                                // negative votes for each validator
mapping(address => bytes32) validationHash; // val hash
uint64 winnerConfirmations; // number of winner confirmations
address winner;              // winner of this race
uint256 price;               // price for the validation
    }

```

- **mapping(address => mapping(bytes32 => bool)) public votes;**

Keeps record of votes of each validator for each request. A validator can vote only once for other validators to determine the winner.

- **mapping(uint256 => mapping(bytes32 => uint256)) public activeOperatorsStake;**

Tallies amount of total stake of all validators for each validation.

- **uint256 public recentTimestamp;**

Timestamp of the validation preceding the current one, to decide the total number of validators and calculate the quorum.

- **bytes32 public recentValidationHash;**

Validation hash of the validation preceding the current one, to decide the total number of validators and calculate the quorum.

- **uint256 public quorum;**

Current quorum. First validation will be 100% quorum to get the

process going with minimum engagement.

- `mapping(bytes32 => Validation) public validations;`

Tracks each validation

Events

- `event ValidationInitialized(address indexed user, bytes32 indexed validationHash, uint256 initTime, bytes32 documentHash, string url);`

Emitted when validation has been initialized by the requestor.

- `event ValidatorValidated(address indexed validator, bytes32 indexed documentHash, uint256 indexed validationTime, ValidationStatus decision, string valUrl);`

Emitted when the validator has validated the request.

- `event RequestExecuted(address indexed requestor, bytes32 indexed validationHash, bytes32 documentHash, uint256 consensus, uint256 quorum, uint256 timeExecuted, string url, address[] winners);`

Emitted when required quorum has been reached and request is considered validated.

- `event PaymentProcessed(bytes32 validationHash, address winner, uint256 pointsPlus, uint256 pointsMinus);`

Emitted when payment has been processed and the winning validator paid.

- `event WinnerVoted(address validator, address winner, bool isValid);`

Emitted when the validator has voted on the race winner.

Functions

- **function initialize**(
 address _members,
 address _memberHelpers,
 address _nodeOperations,
 address _validationHelpers,
 address _queue) **external virtual**

Initializes all variables for upgradable contracts. Also takes place of a typical constructor.

- **function checkIfRequestorHasFunds**(**address** requestor, **uint256** price)
public view returns (bool)

Will return true if the requester has enough funds to initiate validation based on the price.

- **function initValNoCohort**(**bytes32** documentHash, **string memory** url,
bool isCohort, **uint256** price) **external**

It will create a request to validate, which will trigger Node Operators to run the validation process.

- **function voteWinner**(**address[] memory** winners, **bool[] memory** vote,
bytes32 validationHash) **external nonReentrant**

It will accept votes from validators against other validators.

- **function returnValidatorCount**(**bytes32** _recentValidationHash) **public view returns (uint256)**

It will return the current number of validators for given validation.

- **function returnValidatorList**() **public view returns (address[] memory)**

It will return a list of currently registered validators.

- `function returnValidatorListActual(bytes32 validationHash) external view returns (address[] memory)`

It will return a list of validators currently available for last validation.

- `function collectValidationResults(bytes32 validationHash) public view returns (
 address[] memory,
 uint256[] memory,
 uint256[] memory,
 uint256[] memory,
 string[] memory,
 bytes32[] memory
)`

It will return a list of validators with their stakes, vote results, their times, urls of their reports, and report hashes for given validationHash.

- `function isValidated(bytes32 validationHash) external view returns (ValidationStatus)`

It will return the status of validation for a specific validator who is calling this function and validation hash.

- `function hasVoted(bytes32 validationHash) external view returns (bool)`

It will return true if the validator has voted.

- `function processPayments(bytes32 validationHash, address winner) internal`

It will process the payment once the required quorum of validators have processed the validation.

- `function executeValidation(bytes32 validationHash, bytes32 documentHash, uint256 _quorum) internal`

It is called when required quorum has been reached. When this call occurs, validators can verify each other.

- **function validate**(
 bytes32 documentHash,
 uint256 validationTime,
 address subscriber,
 ValidationStatus decision,
 string memory valUrl,
 bytes32 reportHash) **external virtual nonReentrant**

Can be called by the validator to initiate the process of verification by invoking Pacioli. Results are passed to the function.

- **function returnValidationRecord**(bytes32 validationHash) **external view returns** (
 bool cohort,
 address requestor,
 uint256 validationTime,
 uint256 executionTime,
 string memory url,
 uint256 consensus,
 uint256 validationsCompleted,
 uint64 winnerConfirmations,
 bool paymentSent,
 address winner
)

It will return a validation record based on the validation hash.

- **function returnUrl**(bytes32 validationHash, address user) **external view returns** (string memory url)

It will return a validation url based on the validation hash and user who is calling it.

- **function returnWinnerPoints**(bytes32 validationHash, address user) **external view returns** (uint256 plus, uint256 minus)

```
It will return winner points based on validators voting against each other.
```

RulesERC721Token

Overview

Introduction

This NFT token will represent rules/controls created by validators. A rule creator can submit a rule to be validated by a network of validators. Once the rule is validated (mostly syntactically), it becomes available for claiming as an NFT. Later rule creators can be paid royalties for using their rule within the protocol.

Global variables and types

- `ValidationsNoCohort public validation` - validation contract
- `mapping(bytes32 => bool) public NFTCompleted` - true if claimed

Events

- `event Mint(uint256 tokenId, address recipient);`

Emitted when a new token is minted.

Functions

- `function tokenURI(uint256 tokenId) public view virtual
override(ERC721URIStorageUpgradeable, ERC721Upgradeable) returns
(string memory)`

Returns token uri for given token id.

- **function** **mintTo**(**bytes32** _hash)**public returns** (**uint256**)

It will mint a new token when called by the rightful owner.

It will check if the rule has passed validation and if it has been validated at all.